VHDL Workflow & Logisim Simulation Report

이름: 김민재

학번: 2023029643

학과: 로봇공학과

제출일: 2026. 10. 26

1.개요(Overview)

본 보고서는 Logisim-evolution 환경에서 VHDL 언어를 활용하여 디지털 논리 회로 모듈 5종(assignment1~assignment5)을 설계, 구현 및 시뮬레이션한 과정을 기술한다. 조합 논리 게이트 구현부터 MUX, 7-Segment Display 제어, 반복문 기반 카운터 및 조건부 디스플레이 시스템까지의 구현 결과를 정리하였다.

2. 과제별 상세 구현 (Assignments 1 ~ 5)

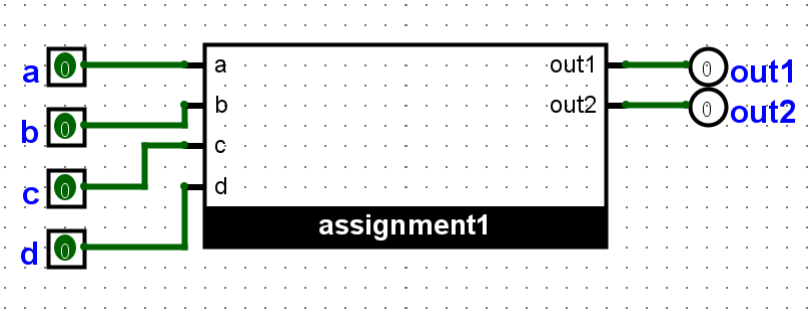
Assignment 1: basic_logic (기본 논리 게이트)

설계 목적: 4개의 1비트 입력(a, b, c, d)에 대한 4-Input AND 게이트 및 OR 게이트 출력 구현

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity assignment1 is
    port(
        a, b, c, d : in  std_logic;
        out1       : out std_logic;
        out2       : out std_logic
    );
end entity assignment1;

architecture bhv of assignment1 is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```



구현 설명: 입력 a, b, c, d가 모두 '1'일 때만 out1이 '1'이 되며, 입력 중 하나라도 '1'이면 out2가 '1'이 됨을 검증함.

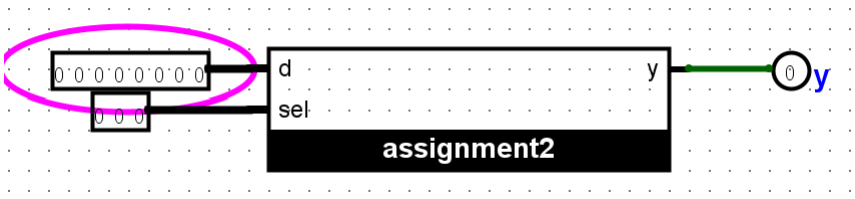
Assignment 2: mux8to1 (8:1 멀티플렉서)

설계 목적: 8비트 데이터 입력 d(7 downto 0)와 3비트 선택 입력 sel(2 downto 0)를 조합한 8:1 MUX 구현

```
library ieee;
use ieee.std_logic_1164.all;

entity assignment2 is
    port(
        d   : in  std_logic_vector(7 downto 0);
        sel : in  std_logic_vector(2 downto 0);
        y   : out std_logic
    );
end entity assignment2;

architecture bhv of assignment2 is
begin
    process(d, sel)
    begin
        case sel is
            when "000" => y <= d(0);
            when "001" => y <= d(1);
            when "010" => y <= d(2);
            when "011" => y <= d(3);
            when "100" => y <= d(4);
            when "101" => y <= d(5);
            when "110" => y <= d(6);
            when "111" => y <= d(7);
            when others => y <= '0';
        end case;
    end process;
end architecture bhv;
```



구현 설명: process 문 내부의 case-when 구문을 사용하여 sel 코드(000~111)에 맞게 해당 인덱스의 입력 비트 d(i)를 출력 y에 전달함.

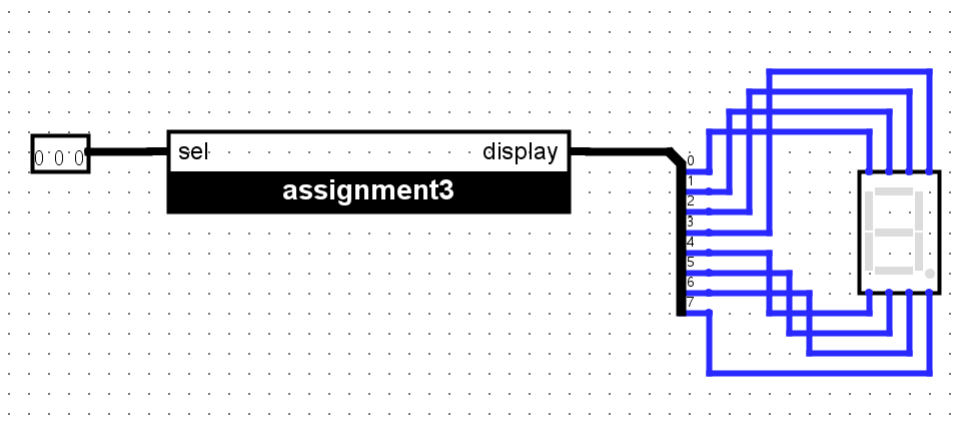
Assignment 3: led (7-Segment 알파벳 표시 모듈)

설계 목적: 3비트 선택 입력(sel)에 따라 지정된 알파벳문자(a, b, c, d, f, g)를 7-Segment Display에 디코딩하여 출력

```
library ieee;
use ieee.std_logic_1164.all;

entity assignment3 is
    port(
        sel     : in  std_logic_vector(2 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end entity assignment3;

architecture bhv of assignment3 is
begin
    process(sel)
    begin
        case sel is
            --  비트 위치:  7  6  5  4  3  2  1  0
            --  담당 세그: dp  C  D  E  B  A  F  G
            when "000" => display <= "01011111"; -- 'a' (1, 2, 3, 4, 6, 0)
            when "001" => display <= "01110011"; -- 'b' (1, 4, 5, 6, 0)
            when "010" => display <= "00110110"; -- 'c' (1, 2, 4, 5)
            when "011" => display <= "01111001"; -- 'd' (3, 4, 5, 6, 0)
            when "100" => display <= "00010111"; -- 'f' (1, 2, 4, 0)
            when "101" => display <= "01101111"; -- 'g' (1, 2, 3, 5, 6, 0)
            when others => display <= "00000000";
        end case;
    end process;
end architecture bhv;
```



구현 설명: 물리적인 Splitter 핀 매핑 규칙[dp, C, D, E, B, A, F, G]에 맞춰 8비트 디스플레이 벡터값을 매핑하여 정확한 문자가 표시되도록 구현함

Assignment 4: vote25 (25인 투표 결과 판독기)

설계 목적: 25비트 투표 입력 vt(25 downto 1) 중 찬성('1')의 개수를 세어 과반수(13명 이상)일 경우 result를 '1'로 출력.

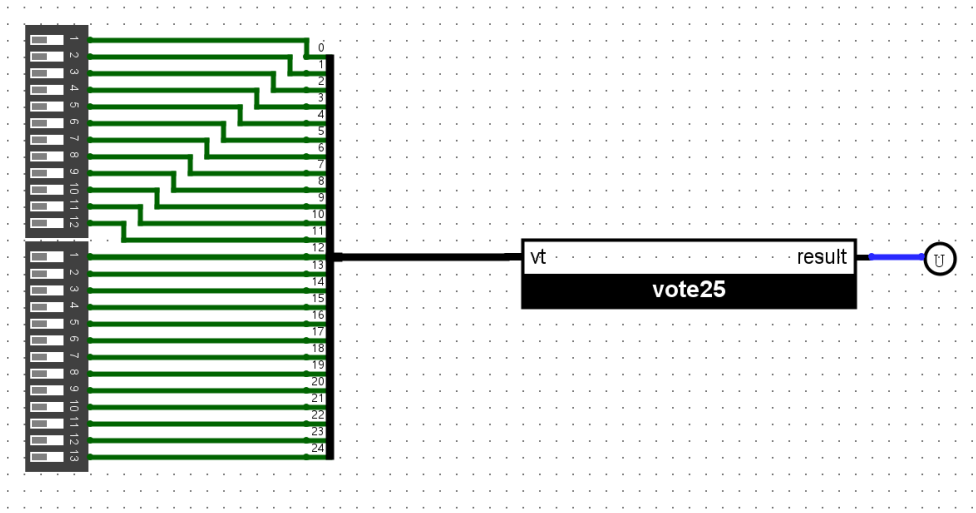
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity vote25 is
    port(
        vt     : in  std_logic_vector(25 downto 1);
        result : out std_logic
    );
end entity vote25;

architecture bhv of vote25 is
begin
    process(vt)
        variable sum: integer range 0 to 25;
    begin
        sum := 0;


        for i in 1 to 25 loop
            if (vt(i) = '1') then
                sum := sum + 1;
            end if;
        end loop;


        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```



구현 설명: process 내의 variable sum과 for loop를 사용하여 25비트 입력의 '1' 개수를 순회 합산하고, 조건문으로 찬성 13표 이상 여부를 판별함. Logisim 회로에서는 DIP Switch 및 Splitter를 통해 25개 입력을 구성함.

Assignment 5: vote13_enable (13비트 카운터 & Enable Display)

설계 목적: en 신호에 따라 13비트 입력 vt(13 downto 1) 중 '1'의 개수를 카운트하여 7-Segment Display에 16진수 형태(0~9, A, b, C, d)로 표시.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

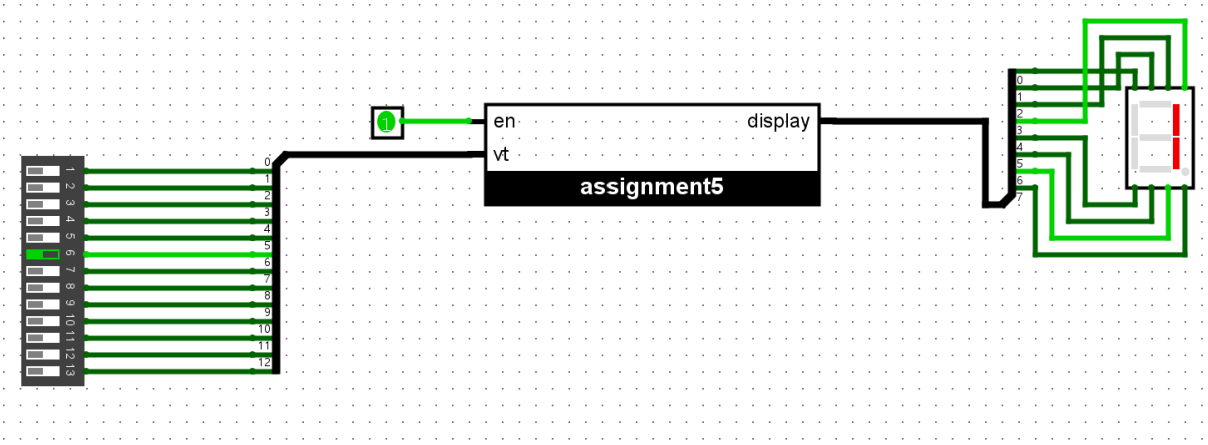
entity assignment5 is
    port(
        en      : in  std_logic;
        vt      : in  std_logic_vector(13 downto 1);
        display : out std_logic_vector(7 downto 0)
    );
end entity assignment5;

architecture bhv of assignment5 is
begin
    process(en, vt)
        variable sum : integer range 0 to 15;
    begin
        if en = '0' then

            display <= "ZZZZZZZZ";
        else

            sum := 0;
            for i in 1 to 13 loop
                if (vt(i) = '1') then
                    sum := sum + 1;
                end if;
            end loop;

            case sum is
                when 0  => display <= "01111110"; -- '0'
                when 1  => display <= "01001000"; -- '1'
                when 2  => display <= "00111101"; -- '2'
                when 3  => display <= "01101101"; -- '3'
                when 4  => display <= "01001011"; -- '4'
                when 5  => display <= "01100111"; -- '5'
                when 6  => display <= "01110111"; -- '6'
                when 7  => display <= "01001110"; -- '7'
                when 8  => display <= "01111111"; -- '8'
                when 9  => display <= "01101111"; -- '9'
                when 10 => display <= "01011111"; -- 'A'
                when 11 => display <= "01110011"; -- 'b'
                when 12 => display <= "00110110"; -- 'C'
                when 13 => display <= "01111001"; -- 'd'
                when 14 => display <= "00110111"; -- 'E'
                when 15 => display <= "00010111"; -- 'F'
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```



구현 설명: en = '0'일 경우 즉시 화면을 끄며, en = '1'일 경우 for loop로 13개 입력 비트의 '1' 개수를 합산해 0부터 13까지의 결과를 7-세그먼트로 표시하도록 구성함.

3. 결론 및 소감

Logisim-evolution 환경에서 VHDL 엔티티와 아키텍처 구조를 설계하고, Splitter 및 DIP Switch, 7-Segment Display와의 상호 연결을 성공적으로 구현하였다. 특히 process 문 내부에서의 variable 활용법, for loop 반복문을 통한 비트 합산 연산, 그리고 7-Segment LED 핀 매핑 구조를 정확히 이해할 수 있었다.